

3교시: 컨테이너 보안 기초

수업 목표

- container 보안을 image, runtime, registry, secret 관점으로 분류한다.
- root 실행, image 출처, tag 고정, secret 관리의 기본 위험을 설명한다.
- public push 전 확인해야 할 gate를 작성한다.

50분 흐름

시간	활동	비중	산출
0-8분	보안 범위 설정	설명 15%	security scope
8-20분	image 출처와 tag 위험	설명 25%	trust note
20-32분	secret과 build context	실행 25%	secret checklist
32-42분	runtime 권한과 root 주의	설명 20%	least privilege note
42-50분	push 전 security gate	실행 15%	push gate

Visual 1: 컨테이너 보안 기초



이 visual은 image trust, secret, root, tag, registry push를 하나의 보안 점검판으로 보여준다.

핵심 설명

컨테이너 보안은 Kubernetes부터 시작되는 것이 아니다. Dockerfile을 작성하고 image를 build하는 순간부터 시작된다. image에 무엇이 들어갔는지, 어디서 온 base image인지, 어떤 tag를 쓰는지, 어떤 secret이 build context에 포함되는지 확인해야 한다.

Day 5 수준의 보안 목표는 취약점 스캐너를 완벽히 쓰는 것이 아니라 "명백한 위험을 만들지 않는 것"이다.

보안 범위

범위	Day 5에서 다루는 것	오늘 다루지 않는 것
Image	base image, tag, copied files	full CVE remediation
Secret	.env, token, key 제외	secret manager 실습
Runtime	root/admin 과용 주의	seccomp/AppArmor 심화
Registry	public push gate	private registry 운영
Host	Docker daemon 권한 인식	host hardening

image 출처와 tag

공식 image라고 해도 tag와 update 정책은 확인해야 한다. nginx:latest는 수업 evidence로 부족하다. 시간이 지나면 같은 tag가 다른 digest를 가리킬 수 있고, 재현성이 약해진다.

좋은 기록:

```
base image: nginx:1.27-alpine
reason: small static web server image, explicit version tag
```

부족한 기록:

```
nginx 썸
```

secret 비노출

secret은 image layer, README, screenshot, terminal history, public registry에 남을 수 있다. 실습 password도 반복적으로 공개하는 습관을 만들면 안 된다.

확인할 파일:

```
find . -maxdepth 2 -type f -print
sed -n '1,120p' .dockerignore
```

위험한 파일 예:

```
.env
id_rsa
aws-credentials
dockerhub-token.txt
prod-password.txt
```

root 실행 주의

많은 official image는 기본 user가 root일 수 있다. Day 5에서는 모든 image를 non-root로 고치는 심화까지 하지 않지만, root 실행이 기본값일 수 있다는 사실과 그 위험은 설명한다.

실무에서는 다음을 검토한다.

질문	이유
container process user는 누구인가	권한 최소화
write 가능한 path는 어디인가	변조/손상 범위
host mount가 write 가능한가	host file 손상 위험
Docker daemon 접근이 필요한가	host 수준 권한 위험

public push gate

public registry에 push하기 전 최소 확인:

Gate	질문
Content	image에 secret/private data가 없는가
Tag	식별 가능한 tag인가
License	base/app content 공유 가능성 확인
README	실행 방법과 위험이 문서화됐는가
Account	개인 token/MFA가 노출되지 않았는가

Day 5 기본 실습은 push를 요구하지 않는다. push는 강사가 명시적으로 요청할 때만 한다.

학술 기준 연결

NIST NICE 관점에서는 credential handling과 least privilege가 핵심이다. ABET의 전문적 책임 관점에서는 학생이 "실행됐다"보다 "안전하게 공유 가능한가"를 판단해야 한다.

실무 failure mode

Failure mode	영향	예방
.env COPY	credential leak	.dockerignore
public push 실수	data exposure	push gate
latest tag만 기록	rollback/reproduce 어려움	explicit tag
root 권한 과신	침해 영향 확대	least privilege
screenshot에 token 노출	credential leak	masking

오해 점검

오해	교정
로컬 실습 password는 공개해도 된다	습관이 중요하며 공개 repository에는 피한다
image build가 되면 안전하다	build 성공과 보안은 다르다
official image는 무조건 안전하다	출처는 좋지만 tag/update/CVE는 별도 판단
Docker 보안은 운영팀만 본다	개발/빌드 단계부터 공동 책임이다

기록 템플릿

```
## Container Security Note
- Base image:
- Tag:
- Secret excluded:
- `.dockerignore`:
- Public push decision:
- Root/user note:
- Remaining risk:
```

평가 기준

기준	2점 evidence
secret	image/README/screenshot 노출 위험을 설명했다
image trust	출처와 tag 기준을 설명했다
least privilege	root/admin 과용 위험을 언급했다
push gate	public push 전 확인표를 작성했다

전이 과제

Week 3 MSA에서는 service와 image 수가 늘어난다. 각 service image마다 같은 security gate가 필요하다. 학생은 "서비스가 4개면 secret 노출 경로가 몇 배로 늘어나는가"를 짧게 쓴다.

공식 근거 링크

- Docker security: <https://docs.docker.com/engine/security/>
- Docker Scout policy: <https://docs.docker.com/scout/policy/>

위험 분류표

위험	가능성	영향	Severity	완화
.env image 포함	중간	높음	High	.dockerignore와 review
public registry push	낮음	높음	High	push gate와 instructor approval
unpinned base image	높음	중간	Medium	explicit version tag
root default process	중간	중간	Medium	user 확인과 least privilege
host socket mount	낮음	높음	High	실습에서 사용 금지
token screenshot	중간	높음	High	masking과 crop

Security Gate Worksheet

```
## Security Gate
- Public push requested:
- `.env` present in build context:
- `.dockerignore` excludes secrets:
- README contains real credentials:
- Screenshot contains token/MFA:
- Base image source:
- Tag:
- Decision:
```

least privilege 기초

최소 권한은 "아무 권한도 쓰지 않는다"가 아니라 필요한 권한만 쓰는 것이다. Day 5에서는 다음을 최소 기준으로 삼는다.

- Docker daemon 접근은 필요한 실습 명령에만 사용한다.
- public push는 명시 요청 전 수행하지 않는다.

- host directory mount는 필요한 경로만 사용한다.
- read-only mount를 쓸 수 있으면 read-only를 우선한다.
- secret 값은 image, README, screenshot에 남기지 않는다.

image trust checklist

```
## Image Trust
- Base image source:
- Base image tag:
- Why this image:
- What is copied into it:
- What is intentionally excluded:
- Push decision:
```

secret handling examples

나쁜 예	좋은 예
ENV API_KEY=real-key	runtime secret 주입
README에 password 기재	placeholder와 변수명만 기록
.env commit	.env.example commit
terminal screenshot에 token	token masking
image에 credential file COPY	.dockerignore로 제외

incident scenario

상황: 학생이 실수로 .env를 image에 복사한 뒤 public registry에 push했다.

대응: 1. registry image를 삭제하거나 private 처리한다. 2. 노출된 credential을 폐기하고 재발급한다. 3. Dockerfile과 .dockerignore를 수정한다. 4. README에 secret handling rule을 추가한다. 5. RCA에 timeline, impact, prevention을 기록한다.